reuse_aware on RTX 4070

Reuse-aware CPU/GPU expert-split rule vs FreeToken's q*, live decode on three MoE models, three tasks and three expert-cache sizes

Item	Value
Study	reuse_aware_models / study_20261020T071921Z
Date of runs	20 October 2026, 07:29-09:57 UTC (about 2.5 h of measurement)
Host	RTX 4070 12 GB, PCIe gen4 x4, Ryzen 5 8500G, 30 GB RAM
Rule origin	runtime_q study (Saketh); this study extends it to Qwen and Gemma live
Episodes	324 measured (108 per model) + 81 burn-in, 0 failures

Summary

FreeToken decides, at every decode step and MoE layer, how many of the *m* experts that miss the GPU expert cache are fetched over PCIe (*q*) and how many run on the CPU (*m* - *q*). Its rule, q*, balances the two branches for the current step only. The reuse-aware rule adds one term: a fetched expert stays in the GPU cache and may be used again in later tokens, so a fetch is worth more than one step's work. This study measured both rules live, with greedy decoding, on gpt-oss-20b, Qwen3.6-35B-A3B and Gemma-4-26B-A4B.

- **gpt-oss-20b: the reuse-aware rule is faster.** 1.15x [1.08, 1.22] vs the paper's q*, and 1.58x [1.45, 1.73] vs FreeToken's built-in q*. The gain grows with cache size, reaching 1.41x vs the paper's q* at a cache of 40% of experts.
- **Qwen3.6-35B-A3B: no gain.** 1.00x [0.96, 1.04] vs the paper's q*; 0.96x [0.92, 1.00] vs built-in q*, a small loss.
- **Gemma-4-26B-A4B: no gain.** 0.99x [0.96, 1.02] vs the paper's q*; 1.02x [0.99, 1.05] vs built-in q*.
- **Why:** the rule only fetches when an expert is likely to be routed to again soon. gpt-oss has 32 experts per layer with 4 active, so each expert is reused often. Qwen (256 experts, 8 active) and Gemma (128, 8 active) reuse each expert rarely, so the rule makes almost the same choices as q*.

1. Background: the split decision

In FreeToken's hybrid MoE mode, all expert weights live in host RAM and a GPU cache holds a subset. At each decode step, for each MoE layer, the router picks top-k experts. Those already in the GPU cache run there. For the *m* that miss, the engine picks *q*: q experts are copied over PCIe and run on the GPU, and m - q run on the CPU at the same time. A fetched expert is inserted into the GPU cache (LRU eviction), so fetching is the only way the cache is ever filled during decode.

Both branches run in parallel, so the step's exposed cost in CPU-expert units is

```
C(m, q) = max( q / rho , m - q ),     rho = B_P / B_H
```

where B_P is PCIe gather bandwidth and B_H is CPU MoE bandwidth. On this host rho is about 0.2: running an expert on the CPU is about five times cheaper than moving it over PCIe.

1.1 The three rules compared

Rule	Label in this report	How q is chosen
q* as in the paper	q* (paper)	Fetch fraction = rho, turned into an integer q by FreeToken's own branch-balancing rounding (same code path as the engine); rho from the fresh calibration.
q* as built into FreeToken	q* (built-in)	Engine default (--moe-hybrid-max-fetch auto): fetch a fixed fraction PCIe/(PCIe + CPU) of the misses, from the engine's own overlap benchmark (0.178 on gpt-oss, 0.154 on Qwen and Gemma). Untouched by the hook.
Reuse-aware rule	Reuse-aware	Fetch candidates in decreasing p while R = p * T_res >= Delta(m, q); p is each expert's recent routing frequency in this layer.
Reuse-aware, uniform p	Reuse-aware, uniform p	Same rule with p = top_k / E for every expert (no per-expert routing statistics).

Table 1. Policies. All four use the same engine, kernels, cache and LRU eviction; only q differs.

1.2 The reuse-aware rule

Fetching one more expert changes this step's cost by

```
Delta(m, q) = C(m, q + 1) - C(m, q) = max((q+1)/rho, m-q-1) - max(q/rho, m-q)
```

Delta <= 0 means the fetch hides behind the CPU branch and is free (q*'s whole regime). Delta > 0 is the extra time paid now. In return the expert stays resident and saves one CPU execution each time it is routed to again. The expected number of such reuses is

```
R = p * T_res
  p     = smoothed recent routing frequency of the expert in this layer
          (f + p0 * 8) / (n + 8), prior p0 = top_k / E
  T_res = 32 tokens if the cache has a free slot, else the age (tokens) of the LRU victim
```

The rule fetches the highest-p candidates while R >= kappa * Delta (kappa = 1). It reduces to q* when R = 0, so it can only differ from q* when experts are reused. It runs inside an experiment hook (`experiments/runtime_q/instrumentation/runtime_q_hook.py`, class `ReuseAware`) that the engine loads via FREETOKEN_MOE_RUNTIME_Q_MODULE.

2. Setup

2.1 Hardware and software

Component	Value
GPU	NVIDIA GeForce RTX 4070, 12 GB (12282 MiB), driver 595.84
PCIe link	gen4 x4 (measured gather bandwidth B_P about 6.7 GB/s)
CPU	AMD Ryzen 5 8500G, 6 cores / 12 threads, AVX-512 BF16; 5 threads pinned to cores 0-4 for the CPU MoE executor
Host RAM	30 GB (32.6 GB raw); 4 GB swap, server scope run with MemorySwapMax=0
OS	Linux 7.0.0-31-generic x86_64, glibc 2.39
CUDA / toolchain	CUDA 13.0 (nvcc V13.0.88), torch 2.11.0+cu130, transformers 5.16.1, Python 3.13.13
Engine	FreeToken from .runtime/freetoken with runtime/patches/experiment-hooks.patch; repo at commit 05b1d43 plus uncommitted driver changes (per-group serve args, env and prompt file)
Execution mode	Eager (not CUDA graphs), batch size 1, overlap scheduling disabled

Table 2. Host and software. Taken from the run's environment.json.

2.2 Models

Model	Quant	MoE layers L	Experts E	Active k	Expert size	Expert bank in RAM	k / E
gpt-oss-20b	MXFP4	24	32	4	13.24 MB	10.2 GB	0.125
Qwen3.6-35B-A3B	NVFP4	40	256	8	1.78 MB	18.2 GB	0.031
Gemma-4-26B-A4B	NVFP4	30	128	8	3.35 MB	12.9 GB	0.062

Table 3. Model geometry from each checkpoint's config.json. k / E is the chance that a random expert is routed to on a given token, the uniform p used by one rule variant. Qwen and Gemma also have a shared or dense path that always runs on the GPU and is not part of the split.

All three checkpoints are local Hugging Face directories under results/checkpoints/. Qwen and Gemma were run with --text-model-only (vision towers not loaded). These are the three models whose expert banks fit in this host's RAM.

2.3 Tasks and prompts

Task id	Task	Prompt (abridged)	Source
0	code	Write a Python function that merges two sorted streams of timestamped events, stable on ties, constant extra space, explain complexity, short example.	experiments/prompts/routing_workloads.jsonl #0
1	summarization	Summarize an incident (timeouts after a worker-count change, rising memory, two restarts) for an engineering handoff in three paragraphs; facts vs causes; two follow-up checks.	routing_workloads.jsonl #1
2	math	A sequence obeys a_1 = 2 and a_(n+1) = 3 a_n + 1. Derive a closed form for a_n.	pruning_requests.jsonl math_011

Table 4. The three tasks, stored together in experiments/prompts/reuse_aware_tasks.jsonl. Each prompt goes through the model's own chat template.

The tasks are chosen to produce different routing patterns (code, prose and symbolic reasoning), not to measure answer quality. No accuracy is scored; the output text hash is recorded to check that policies do not change what the model generates.

2.4 Calibration

Bandwidths were measured fresh before the runs with the engine's benchmark (`ft bench bw --dtype nvfp4,mxfp4_triton --model gpt-oss-20b,qwen3.6-moe,gemma4-26b --gpu 0 --cpu-threads` 5), stored in calibration/benchbw.json, and passed to every server via FREETOKEN_BENCHBW_PATH. The rule reads B_P and B_H from FREETOKEN_MOE_RUNTIME_Q_BP / _BH.

Model	B_P (GB/s, PCIe)	B_H (GB/s, CPU MoE)	rho = B_P / B_H	Built-in q* fetch fraction
gpt-oss-20b	6.70	31.76	0.211	0.178
Qwen3.6-35B-A3B	6.71	31.38	0.214	0.154
Gemma-4-26B-A4B	6.71	36.83	0.182	0.154

Table 5. Overlap-mode bandwidths. Gemma has no per-model NVFP4 entry, so its B_H is the generic NVFP4 figure. Built-in q* uses the engine's own fraction, not rho * m, which is why the two q* versions differ.

2.5 Expert-cache sizes

Model	Cache rates	Slots (approx., rate x L x E)	GPU memory for experts at largest rate
gpt-oss-20b	0.10, 0.20, 0.40	77, 154, 307	4.06 GB
Qwen3.6-35B-A3B	0.05, 0.10, 0.20	512, 1024, 2048	3.64 GB
Gemma-4-26B-A4B	0.10, 0.15, 0.20	384, 576, 768	2.58 GB

Table 6. Cache rate = fraction of all experts the GPU cache can hold. Ranges were chosen per model to fit 12 GB with the KV cache. Gemma at 0.30 (1152 slots) failed to start: "Not enough memory for KV cache".

2.6 Server configuration

```
driver -> ft serve: --backend hybrid --hybrid-fetch -1 --cache-rate <r> --no-graph
    --cache-type naive --max-prefill-length 1024 --cpu-threads 5 --serial-load
  plus per model:
  gpt-oss : --mem-ratio 0.9 --max-seq-len 2048
  Qwen    : --mem-ratio 0.9 --max-seq-len 2048 --text-model-only
  Gemma   : --mem-ratio 0.8 --max-seq-len 1280 --text-model-only
env: FREETOKEN_DISABLE_OVERLAP_SCHEDULING=1 MALLOC_ARENA_MAX=2
     OMP_NUM_THREADS=1 MKL_NUM_THREADS=1 FREETOKEN_BENCHBW_PATH=<calibration>
```

3. Experiment design

Factor	Levels
Model	3: gpt-oss-20b, Qwen3.6-35B-A3B, Gemma-4-26B-A4B
Task	3: code, summarization, math
Cache rate	3 per model (Table 6)
Policy	4: q* (paper), q* (built-in), Reuse-aware, Reuse-aware uniform p
Repeats	3 rounds per cell, every policy once per round
Decode length	128 tokens, greedy (temperature 0); 127 inter-token gaps timed
Total	3 x 3 x 3 x 4 x 3 = 324 measured episodes, plus 3 burn-in episodes per server

Table 7. Full factorial design.

3.1 Protocol per cell

- One fresh `ft serve` process per (model, cache rate, task) cell: 27 servers.
- Three burn-in episodes (fixed 25% fetch) first and discarded, because the first episodes after start-up are slow.
- Each episode = a warm-up generation then a measured generation, both under the episode's policy and both starting from a freshly reset (empty) GPU expert cache, so no policy inherits another's cache.
- Policy order rotates each round (round 0: paper, built-in, rule, uniform; round 2 starts with the rule), so no policy always runs first or last.
- Metric: decode latency in ms/token = time from first to last streamed token / 127. Prefill (TTFT) is excluded.
- Round 0 also logs every per-layer decision (m, q, cache hits, bytes moved, decision time) for Section 5.
- One GPU job at a time; queue launched detached; Judge0 containers paused for the whole study.

3.2 Statistics

Within each round of a cell all four policies ran on the same server, so each rule is compared with each q* as a paired ratio: speedup = q* ms/token / rule ms/token. Speedups are summarised as a geometric mean over the 27 pairs per model (or 9 per cache rate, or 9 per task). The 95% interval is a t-interval on the log ratios, and p is a one-sample t-test of log ratio = 0. Above 1 means the rule is faster.

4. Results

4.1 Mean decode latency

Model	q* (paper)	q* (built-in)	Reuse-aware	Reuse-aware, uniform p	Fastest
gpt-oss-20b	48.7	66.9	43.3	43.7	Reuse-aware
Qwen3.6-35B-A3B	71.4	68.6	71.7	70.2	q* (built-in)
Gemma-4-26B-A4B	46.3	47.6	46.8	46.1	Reuse-aware, uniform p

Table 8. Mean ms/token over 3 tasks x 3 cache sizes x 3 rounds (27 episodes per entry). Lower is better.

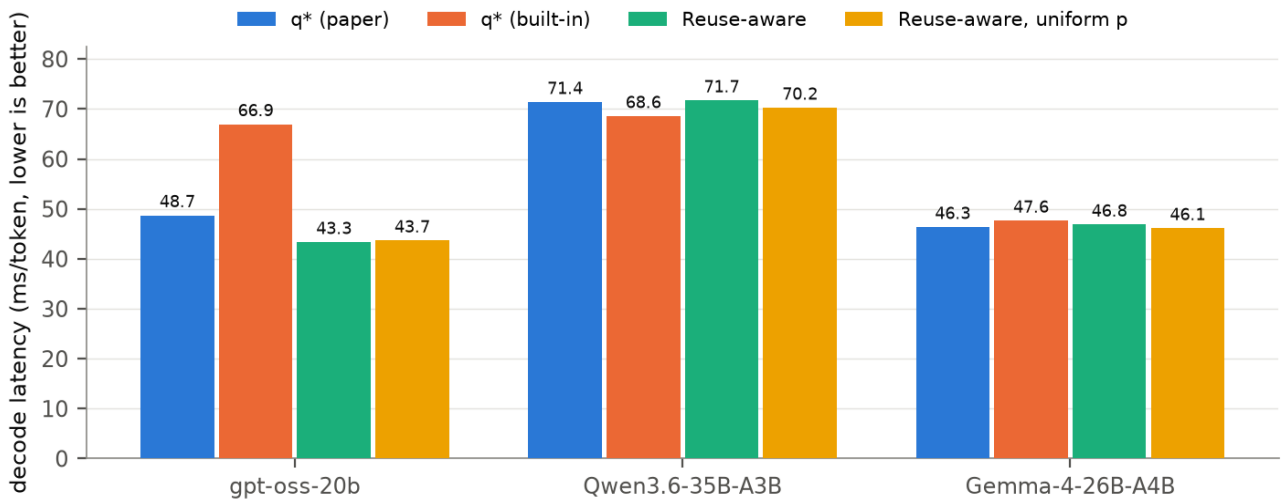


Figure 1. Mean decode latency per model and policy (same data as Table 8).

4.2 Paired speedup of the rule

Model	Rule	vs	Speedup	95% CI	p	Cells faster	Worst / best cell
gpt-oss-20b	Reuse-aware	q* (paper)	1.148x	[1.08, 1.22]	<0.0001	7/9	0.98 / 1.44
gpt-oss-20b	Reuse-aware	q* (built-in)	1.584x	[1.45, 1.73]	<0.0001	9/9	1.20 / 2.19
gpt-oss-20b	Reuse-aware, uniform p	q* (paper)	1.136x	[1.07, 1.20]	<0.0001	7/9	0.98 / 1.40
gpt-oss-20b	Reuse-aware, uniform p	q* (built-in)	1.568x	[1.44, 1.71]	<0.0001	9/9	1.23 / 2.13
Qwen3.6-35B-A3B	Reuse-aware	q* (paper)	0.997x	[0.96, 1.04]	0.8863	4/9	0.92 / 1.15
Qwen3.6-35B-A3B	Reuse-aware	q* (built-in)	0.957x	[0.92, 1.00]	0.0467	2/9	0.86 / 1.05
Qwen3.6-35B-A3B	Reuse-aware, uniform p	q* (paper)	1.016x	[1.00, 1.04]	0.1177	8/9	0.97 / 1.04
Qwen3.6-35B-A3B	Reuse-aware, uniform p	q* (built-in)	0.976x	[0.94, 1.01]	0.1428	2/9	0.91 / 1.10
Gemma-4-26B-A4B	Reuse-aware	q* (paper)	0.991x	[0.96, 1.02]	0.5360	4/9	0.92 / 1.06
Gemma-4-26B-A4B	Reuse-aware	q* (built-in)	1.018x	[0.99, 1.05]	0.2338	6/9	0.93 / 1.11
Gemma-4-26B-A4B	Reuse-aware, uniform p	q* (paper)	1.006x	[0.98, 1.04]	0.6860	4/9	0.94 / 1.08
Gemma-4-26B-A4B	Reuse-aware, uniform p	q* (built-in)	1.034x	[1.00, 1.07]	0.0686	6/9	0.96 / 1.13

Table 9. Geometric-mean paired speedup over 27 pairs per row. Highlighted rows: the whole 95% interval is above 1 (rule reliably faster). Cells faster = how many of the 9 (cache, task) cells have mean speedup above 1.

4.3 By cache size

Model	Cache rate	Rule vs q* (paper)	95% CI	Rule vs q* (built-in)	95% CI	Uniform p vs q* (paper)
gpt-oss-20b	0.10	1.012x	[0.99, 1.03]	1.248x	[1.20, 1.30]	1.010x
gpt-oss-20b	0.20	1.062x	[1.00, 1.13]	1.512x	[1.43, 1.60]	1.059x
gpt-oss-20b	0.40	1.407x	[1.38, 1.44]	2.106x	[2.05, 2.17]	1.370x
Qwen3.6-35B-A3B	0.05	1.077x	[0.99, 1.17]	1.016x	[0.93, 1.11]	1.028x
Qwen3.6-35B-A3B	0.10	0.970x	[0.92, 1.02]	0.970x	[0.93, 1.01]	1.012x
Qwen3.6-35B-A3B	0.20	0.950x	[0.90, 1.01]	0.891x	[0.82, 0.97]	1.008x
Gemma-4-26B-A4B	0.10	0.971x	[0.93, 1.01]	0.980x	[0.90, 1.06]	0.996x
Gemma-4-26B-A4B	0.15	0.979x	[0.90, 1.07]	1.031x	[0.98, 1.08]	1.027x
Gemma-4-26B-A4B	0.20	1.023x	[0.98, 1.07]	1.044x	[1.02, 1.06]	0.995x

Table 10. Speedup by cache size (9 pairs each: 3 tasks x 3 rounds).

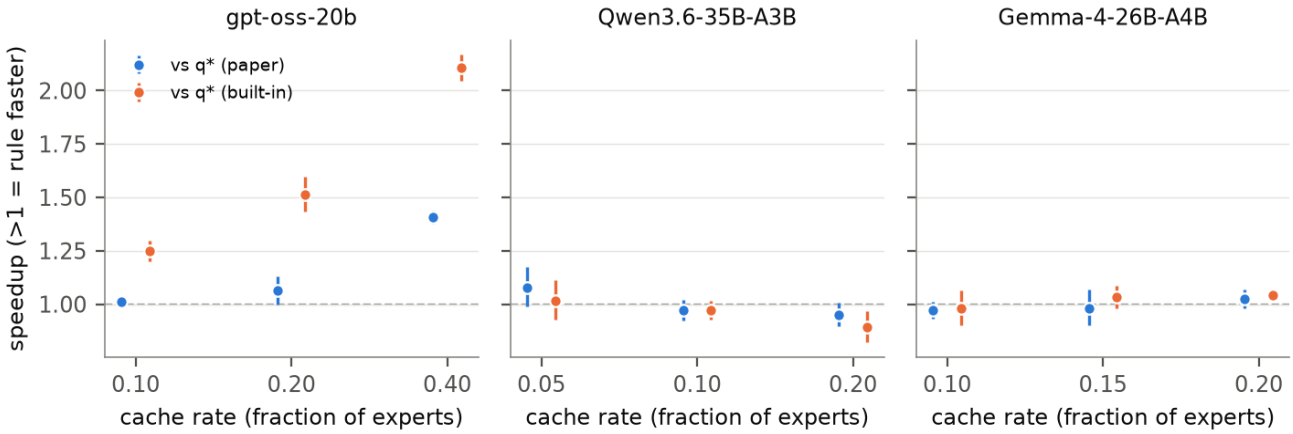


Figure 2. Speedup of the reuse-aware rule (per-expert p) by cache rate, with 95% intervals. Dashed line = no difference.

4.4 By task

Model	Task	Rule vs q* (paper)	95% CI	Rule vs q* (built-in)	95% CI	Uniform p vs q* (paper)
gpt-oss-20b	code	1.095x	[0.97, 1.24]	1.503x	[1.25, 1.80]	1.112x
gpt-oss-20b	summarization	1.170x	[1.04, 1.32]	1.586x	[1.33, 1.89]	1.145x
gpt-oss-20b	math	1.180x	[1.04, 1.33]	1.667x	[1.40, 1.98]	1.151x
Qwen3.6-35B-A3B	code	0.997x	[0.91, 1.09]	0.949x	[0.85, 1.06]	1.023x
Qwen3.6-35B-A3B	summarization	1.034x	[0.96, 1.12]	0.977x	[0.91, 1.05]	0.998x
Qwen3.6-35B-A3B	math	0.962x	[0.91, 1.02]	0.946x	[0.89, 1.01]	1.027x
Gemma-4-26B-A4B	code	0.968x	[0.89, 1.05]	1.009x	[0.94, 1.08]	0.975x
Gemma-4-26B-A4B	summarization	0.988x	[0.97, 1.01]	0.986x	[0.93, 1.05]	1.025x
Gemma-4-26B-A4B	math	1.017x	[0.96, 1.08]	1.060x	[1.02, 1.10]	1.018x

Table 11. Speedup by task (9 pairs each: 3 cache sizes x 3 rounds).

4.5 Per-cell latency

Model	Cache	Task	q* (paper)	q* (built-in)	Reuse-aware	Reuse-aware, uniform p
gpt-oss-20b	0.10	code	56.1 ± 4.1	67.8 ± 2.7	56.6 ± 3.6	**55.2** ± 3.4
gpt-oss-20b	0.10	summarization	54.4 ± 1.5	65.4 ± 1.2	**52.6** ± 1.3	52.7 ± 1.3
gpt-oss-20b	0.10	math	51.0 ± 1.6	65.7 ± 1.0	**50.4** ± 3.6	52.0 ± 3.5
gpt-oss-20b	0.20	code	**48.7** ± 0.6	69.5 ± 1.1	49.8 ± 1.5	48.9 ± 1.5
gpt-oss-20b	0.20	summarization	48.7 ± 4.1	68.3 ± 2.7	**44.7** ± 3.8	45.3 ± 3.8
gpt-oss-20b	0.20	math	46.8 ± 4.6	66.9 ± 2.5	**41.3** ± 1.1	41.9 ± 1.1
gpt-oss-20b	0.40	code	44.1 ± 1.9	66.0 ± 2.0	32.5 ± 1.2	**32.4** ± 1.0
gpt-oss-20b	0.40	summarization	44.7 ± 1.7	65.8 ± 1.2	**31.4** ± 1.1	33.1 ± 0.9
gpt-oss-20b	0.40	math	44.0 ± 2.0	66.9 ± 1.8	**30.6** ± 1.2	31.5 ± 1.1
Qwen3.6-35B-A3B	0.05	code	66.6 ± 10.2	65.2 ± 10.4	**62.3** ± 11.8	64.4 ± 9.6
Qwen3.6-35B-A3B	0.05	summarization	72.5 ± 2.3	65.4 ± 10.2	**63.1** ± 6.2	71.5 ± 2.7
Qwen3.6-35B-A3B	0.05	math	64.4 ± 12.8	**61.9** ± 12.8	63.8 ± 12.7	62.3 ± 14.2
Qwen3.6-35B-A3B	0.10	code	76.0 ± 3.4	**71.7** ± 5.5	75.9 ± 2.0	74.9 ± 2.2
Qwen3.6-35B-A3B	0.10	summarization	76.1 ± 2.6	**74.7** ± 2.1	76.8 ± 1.7	75.6 ± 2.7
Qwen3.6-35B-A3B	0.10	math	66.0 ± 12.5	71.4 ± 13.8	71.5 ± 11.5	**64.9** ± 12.8
Qwen3.6-35B-A3B	0.20	code	74.6 ± 9.7	**70.1** ± 12.9	80.3 ± 2.3	73.4 ± 12.8
Qwen3.6-35B-A3B	0.20	summarization	60.0 ± 2.0	**57.6** ± 1.8	61.9 ± 2.5	61.5 ± 1.2
Qwen3.6-35B-A3B	0.20	math	86.2 ± 10.1	**78.9** ± 3.4	89.8 ± 5.6	83.6 ± 3.5
Gemma-4-26B-A4B	0.10	code	**45.7** ± 1.2	48.2 ± 5.1	48.7 ± 6.1	49.0 ± 7.1
Gemma-4-26B-A4B	0.10	summarization	47.3 ± 5.2	44.6 ± 1.4	48.3 ± 6.4	**44.6** ± 1.0
Gemma-4-26B-A4B	0.10	math	**42.9** ± 1.4	44.4 ± 1.3	43.3 ± 1.3	43.0 ± 1.0
Gemma-4-26B-A4B	0.15	code	42.7 ± 1.4	47.5 ± 7.1	46.8 ± 7.0	**42.7** ± 1.9
Gemma-4-26B-A4B	0.15	summarization	**52.5** ± 1.9	53.1 ± 1.8	54.4 ± 1.3	52.6 ± 1.9
Gemma-4-26B-A4B	0.15	math	45.1 ± 8.9	46.7 ± 5.4	42.1 ± 2.0	**41.1** ± 1.1
Gemma-4-26B-A4B	0.20	code	45.5 ± 3.3	44.3 ± 1.9	**43.3** ± 1.1	46.2 ± 5.7
Gemma-4-26B-A4B	0.20	summarization	42.3 ± 1.9	43.9 ± 1.9	41.6 ± 1.7	**41.4** ± 1.3
Gemma-4-26B-A4B	0.20	math	53.1 ± 2.2	55.8 ± 1.5	**53.0** ± 1.1	54.4 ± 1.2

Table 12. Mean ± standard deviation of ms/token over 3 rounds for every cell. Bold = fastest policy in the cell.

5. What each policy did

These numbers come from the per-layer decision logs of round 0 (every decode step and MoE layer, measured phase), averaged over the 9 cells per model.

Model	Policy	Mean m (misses)	Mean q (fetches)	Cache hit rate	Fetches / token	CPU MB / token	PCIe MB / token	Decision time (us)
gpt-oss-20b	q* (paper)	2.32	0.07	0.42	1.6	715	21	186
gpt-oss-20b	q* (built-in)	4.00	0.00	0.00	0.0	1271	0	164
gpt-oss-20b	Reuse-aware	1.66	0.15	0.59	3.7	477	48	179
gpt-oss-20b	Reuse-aware, uniform p	1.80	0.11	0.55	2.5	538	34	181
Qwen3.6-35B-A3B	q* (paper)	4.25	0.62	0.47	25.0	258	44	340
Qwen3.6-35B-A3B	q* (built-in)	4.85	0.33	0.39	13.0	321	23	325
Qwen3.6-35B-A3B	Reuse-aware	4.11	0.70	0.49	28.1	242	50	337
Qwen3.6-35B-A3B	Reuse-aware, uniform p	4.29	0.56	0.46	22.3	265	40	341
Gemma-4-26B-A4B	q* (paper)	3.41	0.21	0.57	6.4	321	22	222
Gemma-4-26B-A4B	q* (built-in)	3.96	0.11	0.51	3.3	387	11	224
Gemma-4-26B-A4B	Reuse-aware	3.00	0.37	0.62	11.2	264	38	225
Gemma-4-26B-A4B	Reuse-aware, uniform p	3.26	0.27	0.59	8.1	301	27	213

Table 13. Mean per (step, layer) for m and q; per token (all layers) for traffic. Decision time is the median wall time of the hook per decision; it is similar for all policies because the hook gathers the same features under each.

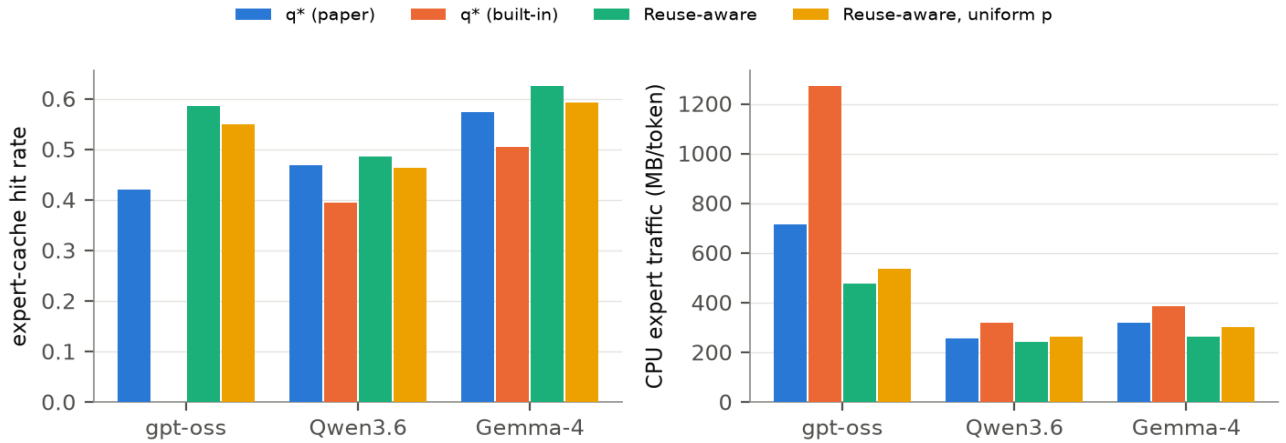


Figure 3. Cache hit rate and CPU expert traffic per model and policy (Table 13).

6. Analysis

6.1 gpt-oss-20b: reuse pays

Built-in q* never fetches on this model: with its fetch fraction of 0.178 and at most 4 misses per layer, the engine's branch-balancing rounding always gives q = 0 (with the paper's 0.211, m = 4 rounds to q = 1). The GPU cache therefore never fills, the hit rate stays at 0 and every routed expert runs on the CPU (1271 MB of CPU expert traffic per token). This reproduces the Part I finding that q* is degenerate on this host. The paper's q* (rho = 0.211) fetches occasionally and reaches a 42% hit rate. The reuse-aware rule fetches about twice as often again, reaches 59% and cuts CPU traffic to 477 MB/token, for 1.15x over paper q* and 1.58x over built-in q*.

The gain depends on cache size: 1.01x at 10%, 1.06x at 20% and 1.41x at 40% vs paper q*. With a small cache the LRU victim is young, so T_res and R are small and the rule fetches little; with a large cache fetched experts survive long enough to be reused many times. All three tasks benefit (1.10-1.18x).

6.2 Qwen and Gemma: the rule and q* coincide

Fetches that hide behind the CPU branch (Delta <= 0) are taken by every rule. Beyond that point each extra fetch costs between 1 and 1/rho = 5 CPU-expert units, so an expert needs p * T_res of that size. With p near top_k/E = 0.03 (Qwen) or 0.06 (Gemma) and T_res of a few tens of tokens, few experts pass, while on gpt-oss (p = 0.125 and higher for hot experts) many do. The decision logs confirm this: the rule's mean q, hit rate and traffic sit within a few percent of the paper's q* (Qwen hit rate 0.49 vs 0.47; Gemma 0.62 vs 0.57). With almost the same decisions, latency is the same within noise, and the confidence intervals all include 1.

On Qwen, built-in q* is the fastest policy (68.6 ms/token) even though it fetches least. Qwen experts are small (1.8 MB) and the CPU handles them well; a likely explanation is that extra fetches buy cache hits but each costs PCIe time that is not fully hidden. This is the regime where fetching less is right, and both the paper's q* and the rule fetch slightly too much.

6.3 Per-expert p vs uniform p

Using each expert's measured routing frequency instead of the constant top_k/E made no reliable difference on any model (Table 9). On gpt-oss both variants give about 1.14-1.15x over paper q*. The benefit comes from accounting for reuse at all, not from ranking which expert to fetch. One reason is that the engine fetches misses by its own order, not by the rule's p ranking, so the rule controls how many experts are fetched but not exactly which.

6.4 Paper q* vs built-in q*

The two versions of q* differ because FreeToken's built-in default does not use rho * m. On gpt-oss this matters a lot: paper q* is 1.37x faster than built-in q* (48.7 vs 66.9 ms/token). On Gemma they are close; on Qwen built-in is slightly better. Any comparison with "q*" should say which version it means.

6.5 Output consistency

Every policy produced the same output text in all 3 rounds of every cell (one hash per policy per cell), so the measurements are repeatable. Different policies sometimes produced different text in the same cell, because CPU and GPU expert kernels are not bit-identical and different splits change rounding; with greedy decoding a single flipped token changes the rest. To check this does not drive the result, the rule vs paper q* speedup was recomputed using only cells where both wrote identical text:

Model	Cells with identical text	Speedup in those cells	Cells with different text	Speedup in those cells
gpt-oss-20b	5 of 9	1.09x	4 of 9	1.23x
Qwen3.6-35B-A3B	4 of 9	1.01x	5 of 9	0.99x
Gemma-4-26B-A4B	4 of 9	1.01x	5 of 9	0.98x

Table 14. Rule (per-expert p) vs paper q* split by whether the two policies generated identical text. The conclusions hold in the identical-text subset.

7. Limitations

- **One machine.** All results are for an RTX 4070 on PCIe gen4 x4 where the CPU runs experts about 5x faster than PCIe can move them (rho about 0.2). A host with a faster link (rho near or above 1) is a different regime; the earlier m_reduction study under precision-corrected settings (rho 1.0-1.55) found the rule slower on all three models.
- **Hook overhead.** The rule runs in a Python hook in eager mode. In this study all four policies ran with the hook loaded and collecting the same features (Table 13), so the comparison is fair, but absolute ms/token is higher than an unhooked build would give.
- **Fetch order.** The engine chooses which misses to fetch; the rule only sets how many.
- **Gemma cache range.** Limited to 20% of experts by GPU memory; gpt-oss, where the gain appeared at 40%, was the only model tested above 20%.
- **Short decodes.** 128 tokens per episode from a cold cache. Longer generations give the cache more time to warm and may change the size of the effect.

- **Three prompts.** One per task type; routing behaviour on other workloads is not covered.
- **Shared expert.** The Qwen shared expert and Gemma dense path run on the GPU outside the split and are included in ms/token but not in the decision statistics.

8. Conclusion

On this RTX 4070 host, accounting for expert reuse helps only where reuse is common. On gpt-oss-20b (32 experts, 4 active) the reuse-aware rule is reliably faster than both versions of q*, by 1.15x over the paper's rule and 1.58x over FreeToken's built-in rule, rising to 1.41x with a large cache. On Qwen3.6-35B-A3B and Gemma-4-26B-A4B, where each expert is routed to rarely, the rule makes nearly the same decisions as q* and is neither faster nor slower within the 95% intervals. The next step is the same plan on the RTX 4090 host and the other architectures listed in docs/Experiment_Plan.pdf, to see how the picture changes with rho.

9. Reproducing

```
# calibration
ft bench bw --dtype nvfp4,mxfp4_triton --model gpt-oss-20b,qwen3.6-moe,gemma4-26b \
    --gpu 0 --cpu-threads 5
# one model (repeat for qwen, gemma)
EXPERIMENT_OUTPUT_ROOT=<study> bash scripts/run.sh runtime_q run \
    --plan configs/runtime_q/reuse_aware_models/gpt_oss.json \
    --model results/checkpoints/gpt-oss-20b --gpu 0 --decode 128
# analysis
bash scripts/run.sh runtime_q analyze_reuse_aware_models --study <study>
```

Artifact	Path
Plans	configs/runtime_q/reuse_aware_models/{gpt_oss,qwen,gemma}.json (+ _pilot)
Prompts	experiments/prompts/reuse_aware_tasks.jsonl
Rule	experiments/runtime_q/instrumentation/runtime_q_hook.py (ReuseAware)
Driver	experiments/runtime_q/driver.py
Analysis	experiments/runtime_q/analyze_reuse_aware_models.py
Raw episodes	results/reuse_aware_models/study_20261020T071921Z/runtime_q/results/*_episodes.jsonl
Decision logs	same folder, decisions_{gpt_oss,qwen,gemma}.csv
Server logs	same study, runtime_q/logs/serve_*.log
Tables	same study, analysis/*.csv and REPORT.md
Calibration	same study, calibration/benchbw.json

Table 15. Where everything is. results/ is git-ignored; archive the study folder to share it.